

Connect with me → <https://www.linkedin.com/in/sakshamarora9575/>
<https://topmate.io/sakshamarora>

Problem Statement

Problem Statement: Mutual Fund FAQ Assistant (Facts-Only Q&A)

Overview

The objective of this project is to build a **facts-only FAQ assistant** for mutual fund schemes, using **Groww** as the reference product context. The assistant will answer **objective, verifiable queries** related to mutual funds by retrieving information exclusively from **official public sources**, such as AMC (Asset Management Company) websites, AMFI, and SEBI.

The system must strictly **avoid providing investment advice, opinions, or recommendations**. Every response must include a **single, clear source link** and adhere to defined constraints around clarity, accuracy, and compliance.

Objective

Design and implement a lightweight **Retrieval-Augmented Generation (RAG)-based assistant** that:

- Answers **factual queries** about mutual fund schemes
- Uses a **curated corpus of official documents**
- Provides **concise, source-backed responses**

Target Users

- Retail investors comparing mutual fund schemes
- Customer support and content teams handling repetitive mutual fund queries

Scope of Work

1. Corpus Definition

- Select **one Asset Management Company (AMC)**
- Choose **3–5 mutual fund schemes**, ensuring category diversity (e.g., large-cap, flexi-cap, ELSS)
- Collect **15–25 official public URLs**, including:
 - Scheme factsheets
 - KIM (Key Information Memorandum)
 - SID (Scheme Information Document)
 - AMC FAQ/help pages

- AMFI/SEBI guidance pages
 - Statement and tax document download guides
-

2. FAQ Assistant Requirements

The assistant must:

- Answer **facts-only queries**, such as:
 - Expense ratio of a scheme
 - Exit load details
 - Minimum SIP amount
 - ELSS lock-in period
 - Riskometer classification
 - Benchmark index
 - Process to download statements or capital gains reports
 - Ensure:
 - Each response is **limited to a maximum of 3 sentences**
 - Each response includes **exactly one citation link**
 - Each response includes a footer:
“**Last updated from sources: <date>**”
-

3. Refusal Handling

The assistant must **refuse non-factual or advisory queries**, such as:

- “Should I invest in this fund?”
- “Which fund is better?”

Refusal responses should:

- Be **polite and clearly worded**
 - Reinforce the **facts-only limitation**
 - Provide a **relevant educational link** (e.g., AMFI or SEBI resource)
-

4. User Interface (Minimal)

The solution should include a simple interface with:

- A welcome message
 - Three example questions
 - A visible disclaimer:
“**Facts-only. No investment advice.**”
-

Constraints

Data and Sources

- Use **only official public sources** (AMC, AMFI, SEBI)
- Do **not** use third-party blogs or aggregator websites

Privacy and Security

- Do **not** collect, store, or process:
 - PAN or Aadhaar numbers
 - Account numbers
 - OTPs
 - Email addresses or phone numbers

Content Restrictions

- No investment advice or recommendations
- No performance comparisons or return calculations
- For performance-related queries, provide a **link to the official factsheet only**

Transparency

- Responses must be **short, factual, and verifiable**
- Every answer must include a **source link and last updated date**

Expected Deliverables

1. **README Document**
 - Setup instructions
 - Selected AMC and schemes
 - Architecture overview (RAG approach)
 - Known limitations
2. **Disclaimer Snippet**
 - “Facts-only. No investment advice.”

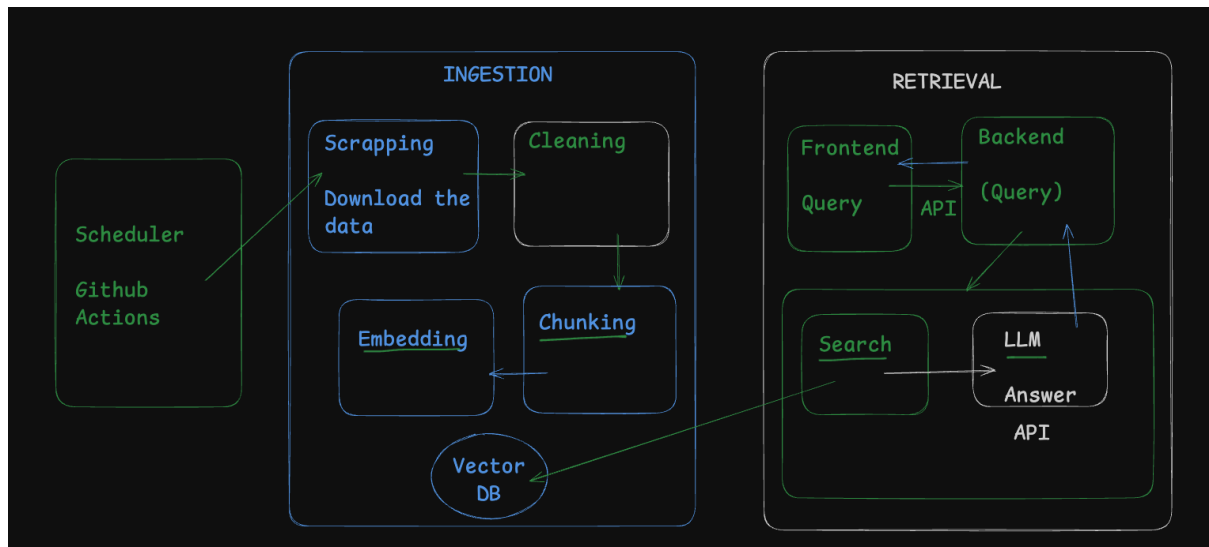
Success Criteria

- Accurate retrieval of factual mutual fund information
- Strict adherence to **facts-only responses**
- Consistent inclusion of **valid source citations**
- Proper refusal of advisory queries
- Clean, minimal, and user-friendly interface

Summary

The goal is to build a **trustworthy, transparent, and compliant mutual fund FAQ assistant** that prioritizes **accuracy over intelligence**. The system should ensure that users receive only **verified, source-backed financial information**, without any advisory bias or speculative content.

Diagram Of Solution



Architecture

Mutual Fund FAQ Assistant — Phase-wise Architecture

Companion design document to problemStatement.md. Goal: a **facts-only**, source-cited, RAG-based Q&A assistant for mutual fund schemes (Groww context). **Iteration scope:** the corpus is strictly the 5 Groww HDFC scheme URLs listed in Phase 0 — no other URLs are ingested or cited.

1. Architectural Principles

These principles drive every phase below:

1. **Facts-over-Intelligence** — retrieval grounds every answer; the **Groq**-hosted generation model only reformats retrieved facts (optional path; default remains extractive).
2. **Single source of truth per answer** — exactly one citation URL per response.
3. **Closed corpus** — only whitelisted official URLs are ingested; nothing else can leak in.
4. **Refusal by default** — advisory / opinion queries are deflected with a polite, educational redirect.
5. **PII-free** — no PAN, Aadhaar, account numbers, OTPs, emails, or phone numbers are collected, logged, or processed.

6. **Determinism > Creativity** — low temperature, strict prompt contracts, hard answer-length caps (≤ 3 sentences).
7. **Auditability** — every response is traceable to a chunk, a document, a source URL, and a "last updated" timestamp.

3. Phase-wise Architecture

The build is structured in **6 phases**, each phase producing a working, demoable artifact.

Phase 0 — Foundation & Governance (Day 0)

Purpose: lock down scope, sources, and guardrails *before* writing code.

Selected AMC: HDFC Mutual Fund (HDFC Asset Management Company Ltd.)

Selected Schemes (5, category-diverse) — these 5 Groww URLs are the *entire* corpus for this project. No other URLs are used.

#	Scheme	Category	Source URL (Groww)
1	HDFC Mid Cap Fund — Direct Growth	Mid Cap	https://groww.in/mutual-funds/hdfc-mid-cap-fund-direct-growth
2	HDFC Equity Fund — Direct Growth	Flexi Cap	https://groww.in/mutual-funds/hdfc-equity-fund-direct-growth
3	HDFC Focused Fund — Direct Growth	Focused	https://groww.in/mutual-funds/hdfc-focused-fund-direct-growth
4	HDFC ELSS Tax Saver — Direct Plan Growth	ELSS	https://groww.in/mutual-funds/hdfc-elss-tax-saver-fund-direct-plan-growth
5	HDFC Large Cap Fund — Direct Growth	Large Cap	https://groww.in/mutual-funds/hdfc-large-cap-fund-direct-growth

Scoping decision (overrides the generic “15–25 URLs” guideline in the problem statement): For this iteration, the corpus is **strictly limited to the 5 Groww scheme pages above**. No AMC PDFs (KIM/SID/factsheets), no AMFI pages, no SEBI pages, no AMC FAQ pages, and no other Groww pages are ingested or cited. Every fact the assistant returns must come from one of these 5 URLs, and every citation must be one of these 5 URLs — verbatim. This is enforced by sources.yaml and by the Phase 3 post-processor’s whitelist check.

Deliverables

- AMC + 5 schemes locked (above).

- sources.yaml containing **exactly these 5 URLs** (no more, no less).
- Refusal taxonomy: intents to refuse (advice, comparison, prediction, recommendation).
- PII deny-list and redaction regex set.

config/sources.yaml — final registry for this iteration

schemes:

- id: hdfc_mid_cap
name: HDFC Mid Cap Fund - Direct Growth
category: Mid Cap
sources:
- url: <https://groww.in/mutual-funds/hdfc-mid-cap-fund-direct-growth>
doc_type: Product_Page

- id: hdfc_equity
name: HDFC Equity Fund - Direct Growth
category: Flexi Cap
sources:
- url: <https://groww.in/mutual-funds/hdfc-equity-fund-direct-growth>
doc_type: Product_Page

- id: hdfc_focused
name: HDFC Focused Fund - Direct Growth
category: Focused
sources:
- url: <https://groww.in/mutual-funds/hdfc-focused-fund-direct-growth>
doc_type: Product_Page

- id: hdfc_elss
name: HDFC ELSS Tax Saver - Direct Plan Growth
category: ELSS
sources:
- url: <https://groww.in/mutual-funds/hdfc-elss-tax-saver-fund-direct-plan-growth>
doc_type: Product_Page

- id: hdfc_large_cap
name: HDFC Large Cap Fund - Direct Growth
category: Large Cap
sources:
- url: <https://groww.in/mutual-funds/hdfc-large-cap-fund-direct-growth>
doc_type: Product_Page

*# Hard rule: any URL not present in this file MUST NOT appear in any answer.
CI compliance gate (Phase 5) fails the build if a generated answer cites a URL
outside this list.*

Citation rule (enforced by Phase 3 post-processor):

1. The cited URL must be **exactly one** of the 5 URLs above (string-equal match against sources.yaml).
2. If no chunk from these 5 URLs supports the query at the confidence threshold → respond *“I don’t have a verified answer for that. Please refer to the official scheme page: <best-matching scheme URL from the 5>.”*
3. For **refusals** of advisory/comparison/prediction queries → the “educational link” returned is the Groww scheme page most relevant to the query (or a generic one of the 5 if no scheme is detected). No off-corpus link (AMFI/SEBI/etc.) is ever returned.

What the assistant explicitly *cannot* answer in this iteration (because the supporting source is not in the corpus):

- Tax-statement / capital-gains download walkthroughs (AMC help pages not ingested).
- Deep regulatory definitions sourced from AMFI/SEBI explainers.
- Anything sourced from KIM/SID PDFs that isn’t already surfaced on the Groww product page.

For these, the assistant returns the *“I don’t have a verified answer”* response with the relevant Groww scheme link.

Components

- config/sources.yaml — the 5-URL registry above; metadata: {scheme_id, doc_type, last_updated, content_hash}.
- config/refusal_intents.yaml — patterns + canned refusal copy (educational link is always one of the 5 Groww URLs).
- config/disclaimer.txt — *“Facts-only. No investment advice.”*

Exit criteria:

- sources.yaml contains exactly 5 entries — the 5 Groww URLs above — and nothing else.
- A reviewer can diff the URL list against the problem statement and confirm 1:1 match.
- Refusal copy and “I don’t know” copy never reference any URL outside the 5.

Phase 1 — Ingestion & Corpus Build (Offline Pipeline)

Purpose: convert the 5 whitelisted Groww URLs into a clean, chunked, embedded, queryable corpus.

Scheduled refresh (latest data): use a **GitHub Actions** workflow with on: schedule: (and optionally workflow_dispatch for manual runs). Each run executes the Phase 1 toolchain (fetch_corpus → extract/clean → chunk_corpus → embed_corpus → build_index, or the Phase 1.7 orchestrator once implemented) so the corpus and index stay aligned with live Groww pages without a dedicated VM cron. Committing refreshed data/ is optional (many teams upload **artifacts** from the workflow instead). Details under **Sub-phase 1.7** below.

URLs → 1.1 Fetcher → 1.2 Extractor → 1.3 Cleaner → 1.4 Chunker → 1.5
Embedder → 1.6 Indexer

|
1.7 Refresh & Health (orchestrates 1.1–1.6)

Phase 1 is broken into **7 sub-phases**, each independently demoable. Each sub-phase has a single, narrow contract — its inputs, outputs, code module, and exit criteria are below. Implementation order is **strictly 1.1 → 1.7**; later sub-phases consume the outputs of earlier ones.

Sub-phase 1.1 — Fetcher

- **Purpose:** pull the 5 Groww HTML pages and persist raw snapshots.
- **Input:** config/sources.yaml (the 5 URLs from Phase 0).
- **Output:** data/raw/<scheme_id>/<timestamp>.html + a sibling meta.json per fetch ({url, fetched_at, http_status, etag, content_hash_raw, fetcher_kind}).
- **Module:** src/mf_faq/ingestion/fetcher.py → Fetcher.fetch_all() -> list[FetchResult].
- **Tech:** httpx for plain HTTP; if extracted text length / required-keyword presence is below threshold, fall back to a headless renderer (playwright).
- **Behavior:**
 - Respect robots.txt before each scheduled run; abort whole run if disallowed.
 - Send If-None-Match (ETag) on subsequent runs; on 304, skip and reuse the previous snapshot.
 - On 4xx/429: stop, alert, retry next scheduled run — never bypass anti-bot.
 - On 301/302 of a whitelisted URL: do **not** auto-follow; raise a governance alert.
- **Edge cases addressed:** EC-1.1, EC-1.2, EC-1.3, EC-1.10.
- **Exit criteria:** all 5 URLs fetched, each data/raw/.../*.html ≥ a minimum byte size, meta.json present, fetcher run reports health=ok for all 5.

Sub-phase 1.2 — Extractor

- **Purpose:** HTML → structured text with **section anchors** (Overview, Scheme Details, Exit Load, Expense Ratio, Min SIP, Riskometer, Benchmark, FAQs).
- **Input:** raw HTML files from 1.1.

Output: data/processed/<scheme_id>/extracted.json:

```
{
  "scheme_id": "hdfc_equity",
  "source_url": "https://groww.in/...",
  "fetched_at": "2026-05-03T10:00:00Z",
  "sections": [
    {"name": "Expense Ratio", "text": "..."},
    {"name": "Exit Load", "text": "..."},
    {"name": "Scheme Details", "text": "..."},
    ...
  ],
  "must_have_anchors": {"Expense Ratio": true, "Exit Load": true, ...},
}
```



```
"extraction_health": "ok"
}
```

-
- **Module:** src/mf_faq/ingestion/extractor.py → Extractor.extract(html, scheme_id) -> ExtractedDoc.
- **Tech:** trafilatura for main-body text + targeted CSS/XPath selectors for each must-have anchor + BeautifulSoup for table cells (expense ratio, exit load).
- **Behavior:**
 - Maintain a per-page **must-have anchors** list; a missing anchor flips extraction_health to degraded and is logged.
 - For image-only facts (riskometer SVG): read alt / aria-label / sibling caption; if all absent, mark not-extracted (no OCR).
 - Run desktop viewport when JS rendering is needed; auto-expand [aria-expanded="false"] accordions.
- **Edge cases addressed:** EC-1.4, EC-1.5, EC-1.7, EC-1.10.
- **Exit criteria:** for all 5 schemes, extracted JSON exists; ≥ 4 of the must-have anchors present per page; extraction_health=ok on the golden snapshot.

Sub-phase 1.3 — Cleaner & Normalizer

- **Purpose:** strip boilerplate, normalize encoding so retrieval tokens match user queries.
- **Input:** extracted JSON from 1.2.
- **Output:** data/processed/<scheme_id>/cleaned.json — same shape as extracted, but each section's text is cleaned.
- **Module:** src/mf_faq/ingestion/cleaner.py → Cleaner.clean(doc) -> ExtractedDoc.
- **Behavior:**
 - Drop known boilerplate ("Mutual fund investments are subject to market risks...", "You may also like", footer links).
 - Unicode NFKC normalization; collapse whitespace; map Rs. / INR → ₹; normalize %, —, —, smart quotes.
 - Strip volatile fields from a separate **stable view** of the doc (NAV, "as on ", today's AUM) to feed the stable content_hash (see 1.7).
 - **Section drop / trim policy** (gatekeeper for what flows into 1.4+):
 - **Drop entirely:** the FAQ section. The extractor still parses the FAQPage JSON-LD as a must-have-anchor health signal, but the Q&A content (mostly Groww UX guidance, "How do I invest...?") is not part of the facts-only corpus.
 - **Trim aggressively:** Fund Manager keeps just the manager's name and tenure (<Initials> <Name> <Joined> - Present); the bio (Education / Experience) is dropped. Fund House keeps the AMC name, rank, total AUM, and incorporation date; phone / email / website / address are dropped.
- **Edge cases addressed:** EC-1.6, EC-1.8, EC-1.12.
- **Exit criteria:** cleaned text contains zero entries from the boilerplate strip-list; cleaned.json contains no FAQ section; the Fund Manager and Fund House sections do not contain bios or contact details; Rs.500 / Rs. 500 / ₹500 all collapse to ₹500.

Sub-phase 1.4 — Chunker

- **Purpose:** section-aware splitting into retrieval units, with full provenance metadata.
- **Input:** cleaned JSON from 1.3.

Corpus reality (recalibrated after 1.3): the 5 trimmed Groww product pages produce 7 sections per scheme, most of them ≤ 100 tokens (Fund Details ≈ 25 , Minimum Investments ≈ 20 , Fund Manager ≈ 15 , Fund House ≈ 30 , Exit Load and Tax ≈ 80). Only **About** is meaningfully long (~ 200 tokens). The original "400–800 token" target makes no sense for this shape: we'd just emit one chunk per section. The behavior below reflects that.

Output: data/processed/<scheme_id>/chunks.jsonl — one JSON per line:

```
{
  "chunk_id": "uuid",
  "scheme_id": "hdfc_equity",
  "scheme_name": "HDFC Equity Fund - Direct Growth",
  "doc_type": "Product_Page",
  "source_url": "https://groww.in/mutual-funds/hdfc-equity-fund-direct-growth",
  "section": "Exit Load and Tax",
  "section_source": "html_section | meta_description",
  "last_updated": "2026-04-15",
  "content_hash": "sha256:...",
  "stable_content_hash": "sha256:...",
  "text": "..."
}
```

- `section_source` is propagated from the cleaner's `Section.source` so retrieval can reason about provenance. `stable_content_hash` is the **document-level** hash from 1.3 (same value on every chunk of a given snapshot) — sub-phase 1.7 uses it to skip re-indexing when only NAV/AUM ticked.
- **Module:** `src/mf_faq/ingestion/chunker/service.py` → `Chunker.chunk(doc: CleanedDoc) -> Iterable[Chunk]`.
- **Behavior:**
 - **Section never spans chunks.** Each chunk's section is exactly one cleaned-doc section name — never a comma-joined list.
 - **One section → one chunk by default.** All sections except *About* fall well under the soft cap (~ 250 tokens) and are emitted whole; this is *good* for fact-shaped queries because each chunk is a self-contained unit.
 - **Soft cap = 250 tokens, hard cap = 400 tokens, overlap = 30 tokens** — only triggers for long sections (today: just *About*). Splits happen on sentence boundaries; never mid-sentence and never mid-numeric-fact (e.g., "₹14,615.19 Cr" must stay together).
 - **No scheme_name prepend in chunk text** — the embedder (1.5) prepends it transiently before vectorization to avoid near-duplicate boilerplate clustering across schemes (EC-1.8). Storing it in text would corrupt downstream display and BM25 tokenization.
 - **Atomic table rows** — when sections like *Fund Details* contain effectively tabular text ("Min. for SIP ₹500 Fund size (AUM) ₹14,615.19 Cr Expense ratio

- 1.11%"), the splitter treats them as one unit even if a future Groww layout grows them.
- **Drop empty / tiny chunks** — if a section's text is < 5 tokens after cleaning (degenerate case), skip it.
- **Edge cases addressed:** EC-1.8 (near-duplicate boilerplate), EC-1.9 (cross-section bleed), EC-2.5 (wrong-section retrieval — section metadata feeds the Phase 2 reranker).
- **Expected output volume: ~7 chunks per scheme → ~35 chunks total across the 5 schemes.** That's an order of magnitude smaller than the original 50–125 estimate, and it's correct for this corpus.
- **Exit criteria:**
 - Each chunk's section is a single value drawn from the cleaned doc's section set.
 - Every chunk's source_url ∈ the Phase 0 whitelist (string-equal match).
 - Total chunk count is in the band $5 \leq n \leq 12$ **per scheme** (current expectation: 7).
 - For each scheme, the union of all chunks' text covers every section name present in cleaned.json (no fact silently dropped).
 - Numeric facts (₹X Cr, X.XX%, X year(s)) are never split across chunks.

Sub-phase 1.5 — Embedder

- **Purpose:** generate dense vectors per chunk.
- **Input:** chunks from 1.4.
- **Output:** data/index/embeddings.parquet (or .npz + chunks.jsonl sidecar) with rows {chunk_id, embedding[float32]} and a sidecar embedder.json: {"model": "bge-small-en", "version": "1.5", "dim": 384}.
- **Module:** src/mf_faq/ingestion/embedder.py → Embedder.embed(chunks) -> EmbeddingBatch.
- **Tech:** bge-small-en via sentence-transformers, OR text-embedding-3-small via the OpenAI API (configurable).
- **Behavior:**
 - Embed f"{scheme_name}\n\n{text}" so near-duplicate boilerplate across schemes still vectors-apart (EC-1.8).
 - Persist the embedder model + version next to vectors; retriever (Phase 2) refuses to load mismatched versions.
- **Edge cases addressed:** EC-1.8, EC-1.11.
- **Exit criteria:** every chunk has exactly one embedding of expected dimension; embedder.json is present; reload round-trip preserves vectors bit-for-bit.

Sub-phase 1.6 — Indexer

- **Purpose:** build the dense + BM25 indexes that Phase 2 queries against.
- **Input:** chunks (1.4) + embeddings (1.5).
- **Output:** data/index/:
 - vector.faiss (or chroma/) — dense index keyed by chunk_id.
 - bm25.pkl — sparse index over chunk text.

- `chunks.jsonl` — canonical chunk store (with metadata).
 - `manifest.json` — `{built_at, embedder, n_chunks, per_scheme_counts, source_hashes}`.
- **Module:** `src/mf_faq/ingestion/indexer.py` → `Indexer.build()`, `Indexer.load()` → `IndexHandle`.
- **Tech:** FAISS or Chroma for dense; `rank_bm25` / Whoosh for sparse.
- **Behavior:**
 - Atomic swap: build into `data/index/.staging/`, then rename. Phase 2 readers see only fully-built indexes.
 - Manifest doubles as the corpus passport — Phase 5 reads it for freshness reporting.
- **Edge cases addressed:** EC-1.11.
- **Exit criteria:** Phase 2 can open the index and run a sample query end-to-end; manifest's `n_chunks` matches the chunk store; per-scheme counts are non-zero for all 5.

Sub-phase 1.7 — Refresh & Health

- **Purpose:** orchestrate 1.1 → 1.6 as a re-runnable pipeline with drift detection and health reporting. This is the only sub-phase that uses **all** the others.
- **Input:** none directly — runs on a schedule.
- **Output:** `data/index/refresh_log.jsonl` — one line per run with per-stage timings, content-hash diffs, anchor health, and outcome $\in \{\text{ok, partial, frozen}\}$.
- **Module:** `src/mf_faq/ingestion/pipeline/service.py` → `Pipeline.refresh(force=False, dry_run=False, skip_fetch=False)`; CLI: `python -m mf_faq.ingestion.refresh (src/mf_faq/ingestion/refresh.py)`.
- **Scheduler (recommended): GitHub Actions**
 - Add a workflow under `.github/workflows/` with on: `schedule`: using UTC cron (e.g. nightly or weekly) to pull **latest** snapshots from the 5 whitelisted URLs and rebuild downstream artifacts.
 - Also expose `workflow_dispatch` so maintainers can refresh on demand without waiting for the next cron tick.
 - Typical job steps: checkout → install Python + deps (`pip install -e "[embed,index]"` or equivalent) → run fetch → extract/clean → chunk → embed → index → optionally upload `data/index/` (and/or `raw/processed`) as **workflow artifacts** or push to a branch / object storage, depending on deployment (Phase 2 consumers read the built manifest).
 - Respect the same governance rules as local runs: robots.txt gate (1.1), no auto-follow on redirects, freeze/drift behavior below.
 - Alternative execution environments (self-hosted runner, VM cron, Airflow) are equivalent triggers — **GitHub-hosted schedule is the default documented choice** for this repo to avoid managing servers.
- **Behavior:**
 - For each URL, compute `stable_content_hash` (volatile fields excluded — see 1.3); only re-chunk + re-embed pages whose stable hash changed.
 - On drift across **≥ 2 URLs in the same window** → freeze the index (don't overwrite), raise a single aggregated alert.

- Soft-404 detection (HTTP 200 but missing must-have anchors) → fail that URL, keep the rest.
- "Last updated from sources" date in chunk metadata is bumped only when stable content actually changed.
- **Edge cases addressed:** EC-1.6, EC-1.10, EC-5.7, EC-5.8.
- **Exit criteria:** dry-run on the bundled snapshot fixtures produces a deterministic refresh_log.jsonl; a synthetic drift across 3 URLs triggers the freeze path with one alert.

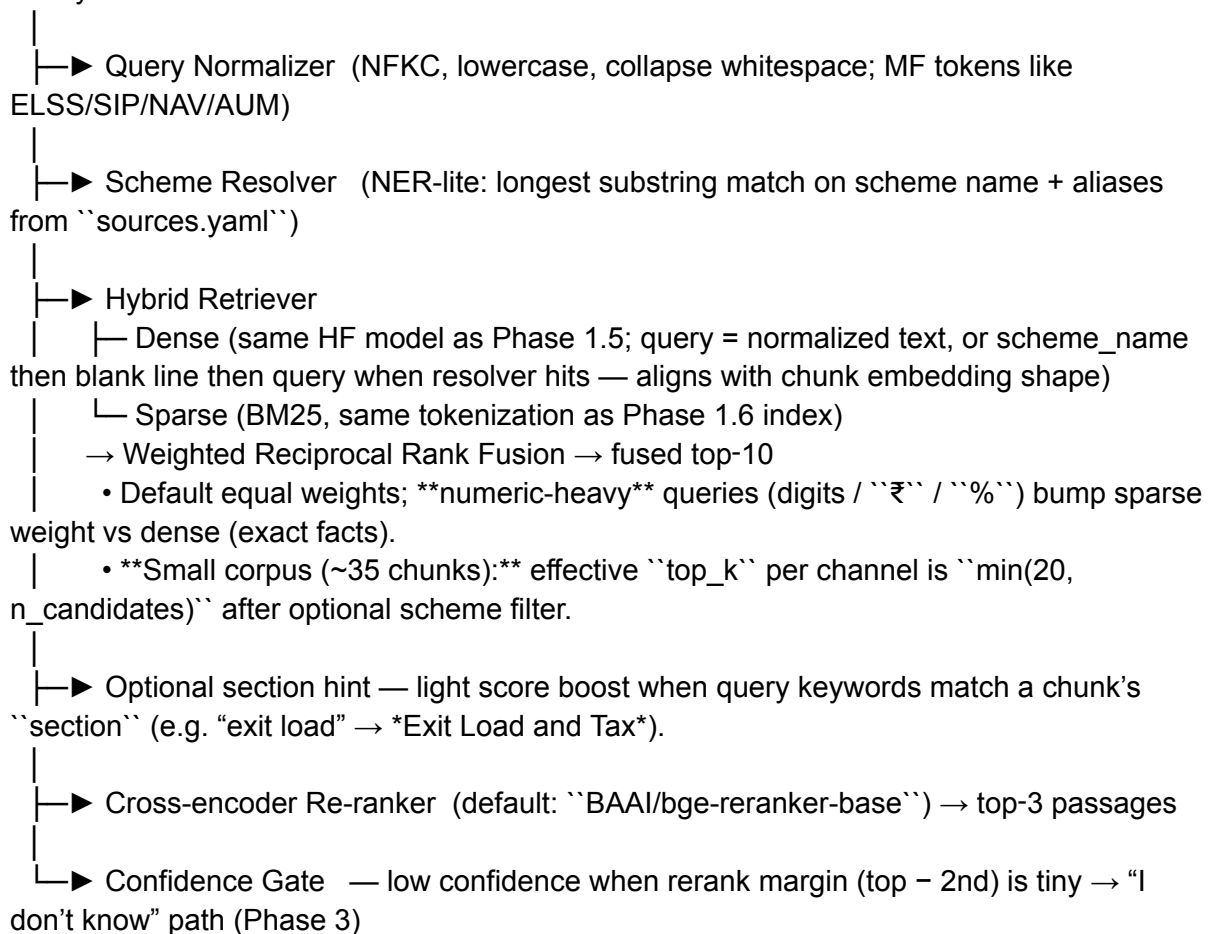
Phase 1 overall exit criteria: the index contains chunks from all 5 Groww URLs (~10–25 per page → ~50–125 total); every chunk's source_url ∈ the 5-URL whitelist; the manifest reports a fully-healthy build; Phase 2 can open and query the index.

Phase 2 — Retrieval Layer

Purpose: given a user query, surface the *minimum* set of chunks needed to answer factually.

Pipeline

Query



Why hybrid + rerank? Mutual fund queries mix exact tokens (e.g., “0.45%”, “1 year”, “₹500”) with semantic phrasing (“how long is the lock-in?”). BM25 nails exact, dense nails semantic; the cross-encoder picks the best chunk.

Filters applied before retrieval (when detectable):

- `scheme_id = <resolved scheme>` → intersect hybrid candidate lists with chunks for that scheme only. If that filters out all candidates (query wording vs snapshot mismatch), **relax** the scheme filter once and search the full corpus (still only the five schemes).
- `doc_type = Product_Page` (only doc type in the corpus for this iteration).

Implementation: `mf_faq.retrieval` — HybridRetriever loads `IndexHandle` + `Embedder` (from `embedder.json`) + cross-encoder; CLI `python -m mf_faq.retrieval "your question"`.

Exit criteria: top-1 chunk contains the gold answer for $\geq 85\%$ of a 30-question eval set.

Phase 3 — Reasoning & Guardrails (Orchestrator)

Purpose: turn a query + retrieved chunks into a compliant, ≤ 3 -sentence answer — or a refusal — with **URL policy** enforced before anything is returned.

Generation stack: when an LLM is used for answer wording (instead of extractive synthesis), it is **Groq** via the OpenAI-compatible Chat Completions API (pip install “mf-faq[groq]”, env `GROQ_API_KEY`). **Default behavior:** `use_groq=None` (auto) — Groq runs when `GROQ_API_KEY` is set; use `use_groq=False` or unset the key to stay extractive-only. Embeddings for retrieval stay as in Phase 1.5 (sentence-transformers and optionally OpenAI embeddings — unchanged).

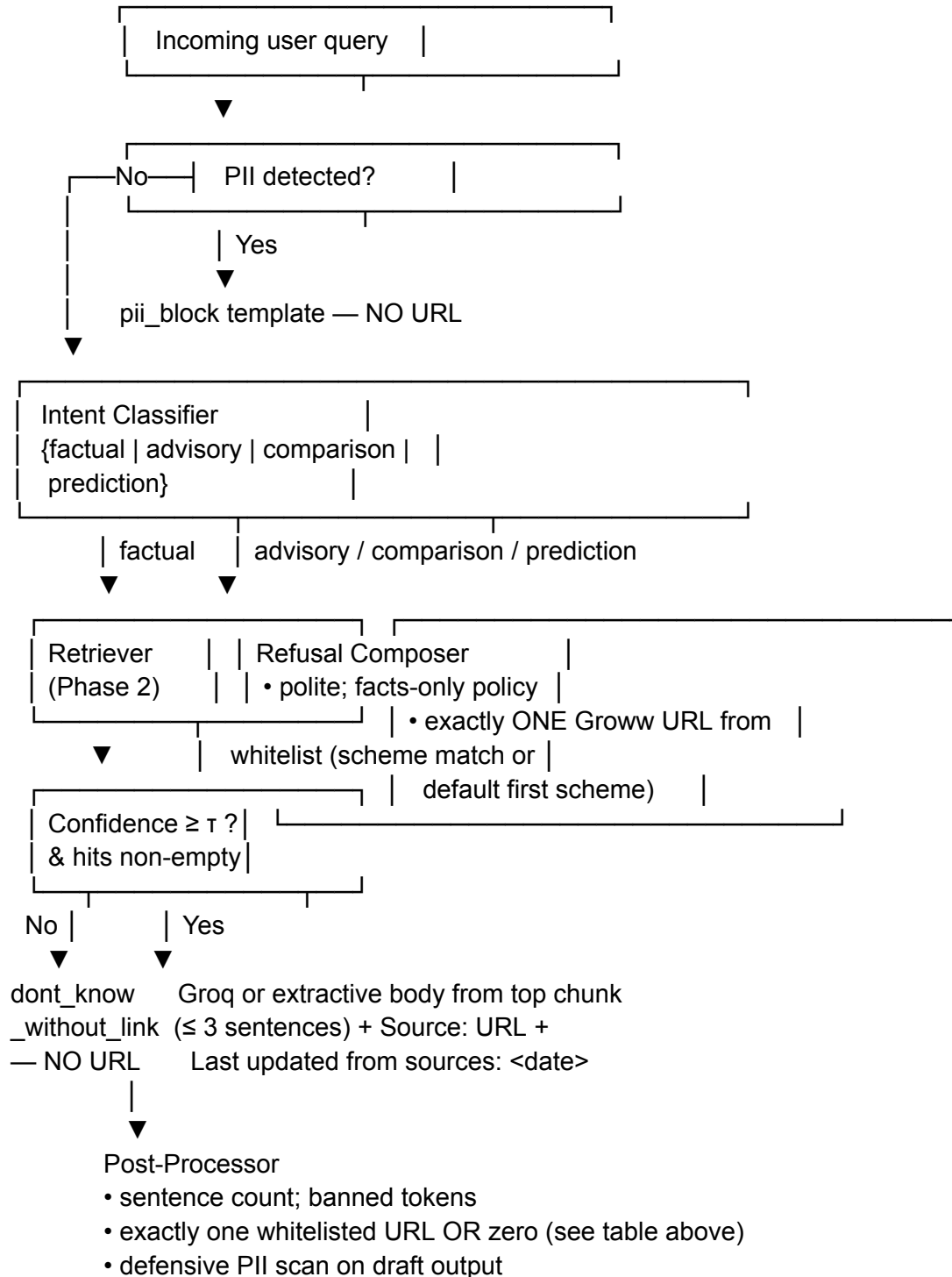
URL policy (non-negotiable)

Situation	URLs in the assistant reply
PII detected in the user message (PAN, Aadhaar, email, phone, OTP, etc.)	None — use the locked <code>pii_block</code> template only.
Insufficient evidence / low confidence / empty retrieval (“don’t know” path)	None — use <code>dont_know_without_link</code> only (ask user to name a scheme; no Groww link).
Non-factual intent (advisory / comparison / prediction refusal)	At most one — the matching scheme’s Groww URL from <code>sources.yaml</code> , or the first scheme’s URL if none resolved.

Successful factual answer
from retrieved chunks

Exactly one — the citation source_url from the top
chunk (must be on the whitelist).

Decision flow



Generator contract

- **Extractive (default):** build the body from the **top reranked chunk** only: first ≤ 3 sentences, strip any URLs embedded in chunk text so the reply does not accidentally contain extra links.
- **Groq (optional):** `Orchestrator(..., use_groq=None)` (auto: Groq if `GROQ_API_KEY` is set), or `use_groq=True` to force the Groq path when a key is present, or `use_groq=False` for extractive-only (package extra [groq]). The model returns **answer body only**; **Source** and **Last updated** lines are appended in code so exactly one whitelisted URL appears. On API/import failure or empty body → extractive fallback.

Groq governance: low temperature; same URL rules as extractive — **no URL** on PII block or don't-know; **exactly one whitelisted URL** on successful factual answers; refusals keep **one** educational URL.

Hard post-checks (deterministic, non-LLM):

- **Route-aware URL count:** PII and insufficient-evidence replies must contain **zero** http(s) URLs. Factual answers must contain **exactly one** URL, and it must appear in `sources.yaml`.
- Sentence count ≤ 3 for the factual body (before the `Source:` line).
- No banned tokens in the full draft (e.g., *"recommend"*, *"should invest"*, *"better than"*, *"will outperform"*).
- Footer present on successful factual path: Last updated from sources: `<YYYY-MM-DD>`.

If post-checks fail on an otherwise retrieved answer → fall back to safe template **with** one whitelisted link (`safe_template`).

Exit criteria: 100% of generated answers respect the URL policy above on the eval set; 0 hallucinated or extra URLs on factual paths.

Phase 4 — User Interface (Minimal Web App)

Purpose: give a clean, trustworthy surface to the assistant.

Layout

Mutual Fund FAQ Assistant	
Facts-only. No investment advice.	[disclaimer]
Welcome! Ask a factual question about <AMC> schemes.	
Try one of these:	
• What is the expense ratio of <Scheme A>?	
• What is the exit load of <Scheme B>?	

• What is the lock-in period for an ELSS fund?

[type your question...] [→]

Answer area

– short answer (≤3 sentences)

– Source: <single link>

– Last updated from sources: <date>

Stack: FastAPI in `mf_faq.ui` exposes `POST /ask`, `GET /meta`, `GET /health`, and serves a minimal static SPA from `mf_faq/ui/static/` at `/` (same-origin fetch). No login, no cookies beyond session, no analytics that capture query text with PII.

UI rules

- Disclaimer always visible.
- Submit button disabled while a query is in flight.
- Answer area renders the citation as a clickable link with `rel="noopener nofollow"`.
- "Copy answer" optional; "Share" deliberately omitted to discourage misuse.

Exit criteria: end-to-end demo: type question → get compliant answer with link + date.

Phase 5 — Evaluation, Compliance & Observability

Purpose: prove the system is accurate, safe, and stays that way.

5a. Evaluation harness

Suite	What it checks	Pass bar
Factual Q&A (30+ Qs)	Exact-match / numeric tolerance vs gold answer	≥ 90%
Citation correctness	Cited URL actually contains the fact	100%
Refusal suite (15+ Qs)	Advice/comparison/prediction queries get refused with educational link	100%

Out-of-corpus	Query about unknown scheme → "I don't have a verified answer"	100%
PII probes	Inputs with PAN/Aadhaar/email are rejected/redacted	100%
Length & format	≤ 3 sentences, 1 citation, footer present	100%

Tooling: a YAML test set + a small pytest runner that calls the API and asserts.

5b. Compliance checks (CI gate)

- Every URL in any answer $\in$ sources.yaml.
- No banned advisory tokens in any answer.
- No PII tokens stored in logs (log-line scanner).

5c. Observability

- Structured logs: {request_id, intent, retrieved_chunk_ids, confidence, post_check_passed, latency_ms}.
- **No raw queries with PII** persisted; queries are hashed for analytics.
- Dashboard (lightweight): refusal rate, "I don't know" rate, top schemes asked about, ingestion freshness.

5d. Operational runbook

- Source change detection → alert when any of the 5 Groww URLs 404s or content hash drifts > X%.
- Weekly re-index job; manual override flag per source.

Exit criteria: all eval suites green in CI; runbook published.

4. Component Inventory (Cross-Phase)

Layer	Component	Responsibility	Phase
Governance	sources.yaml	Whitelist of allowed URLs	0
Governance	refusal_intents.yaml	Refusal patterns + canned copy	0
Ingestion	Fetcher	HTTP/PDF download, ETag/hash tracking	1
Ingestion	Extractor + Cleaner	HTML/PDF → clean text, table-aware	1

Ingestion	Chunker	Heading-aware semantic chunking	1
Ingestion	Embedder	Vector generation	1
Storage	Vector store	Dense ANN search (FAISS/Chroma)	1–2
Storage	Keyword index	BM25	1–2
Storage	Metadata store	Chunk metadata, source registry	1–2
Retrieval	Query normalizer	Acronym expansion, lowercase	2
Retrieval	Scheme resolver	Detect scheme to filter metadata	2
Retrieval	Hybrid retriever	Dense + BM25 + RRF	2
Retrieval	Re-ranker	Cross-encoder for precision	2
Orchestrator	PII guard	Reject inputs containing PII	3
Orchestrator	Intent classifier	factual vs advisory vs comparison etc.	3
Orchestrator	Confidence gate	Trigger "I don't know" on low retrieval score	3
Generation	Groq LLM caller	Templated, low-temp chat completion (optional vs extractive)	3
Generation	Post-processor	Length, citation, banned-token, PII checks	3
Generation	Refusal composer	Polite refusal + educational link	3
UI	FastAPI /ask API	JSON query → orchestrator result + structured logs	4
UI	Static SPA + CSS/JS	Welcome, examples, disclaimer, answer view, copy	4

Quality	Eval harness	Factual + refusal + format + PII suites	5
Quality	Compliance CI gate	Whitelist + banned-token + PII log scan	5
Ops	Refresh scheduler	GitHub Actions schedule (cron) + optional workflow_dispatch; nightly/weekly re-ingest + diff detection	1, 5
Ops	Observability	Structured, PII-free logs + dashboard	5

5. Data Flow — Single Query End-to-End

1. User types: "What is the exit load of HDFC Equity Fund Direct Growth?"
2. UI → POST /ask {query}
3. PII guard → clean
4. Intent classifier → factual
5. Query normalizer → "what is the exit load of hdfc equity fund direct growth"
6. Scheme resolver → scheme_id = hdfc_equity
7. Hybrid retriever → top-10 chunks from
<https://groww.in/mutual-funds/hdfc-equity-fund-direct-growth>
8. Re-ranker → top-1 chunk: "Scheme Details / Exit Load" section
9. Confidence gate → score $0.82 \geq \tau$ → proceed
10. Groq generator (or extractive) → ≤ 3 sentence answer using only that chunk
11. Post-processor → length OK, 1 URL OK, URL $\in$ 5-URL whitelist OK, no banned tokens, footer appended
12. Response:
 "HDFC Equity Fund Direct Growth charges an exit load of 1% if units are redeemed within 1 year of allotment; no exit load applies thereafter.
 Source: <https://groww.in/mutual-funds/hdfc-equity-fund-direct-growth>
 Last updated from sources: 2026-04-15"
13. Logs: {request_id, intent=factual, scheme_id=hdfc_equity, chunk_ids=[...], conf=0.82, checks=passed, latency_ms=740}
 (no raw query stored)

6. Risks & Mitigations

Risk	Mitigation
------	------------

Groww updates a page → stale answer	Content-hash diff in nightly job; bump last_updated; alert if drift big
Groq model hallucinates a number	Numeric values must appear verbatim in retrieved chunk (regex check)
Groq model emits a non-whitelisted URL	Post-processor rejects + falls back to safe template
User asks for advice	Intent classifier + refusal composer (link = matching Groww scheme URL)
User pastes PAN/Aadhaar/email/phone	PII guard rejects request before retrieval; never logged
Ambiguous scheme name	Scheme resolver asks clarifying question (still facts-only, no opinions)
Performance/return question	Redirect to the relevant Groww scheme URL; never compute or compare returns
Low-confidence retrieval	"I don't have a verified answer" + matching Groww scheme URL
Fact only present in KIM/SID (off-corpus)	Return "I don't have a verified answer" + matching Groww scheme URL

7. Phase Roadmap (Suggested Timeline)

Phase	Outcome	Indicative effort
0	Sources whitelisted, refusal taxonomy ready	0.5 day
1	Corpus ingested + indexed	1.5 days
2	Hybrid retrieval + reranker working	1 day
3	Orchestrator + guardrails + generator	1.5 days
4	Minimal UI wired end-to-end	0.5 day

5	Eval suites + CI gates + observability	1 day
---	--	-------

8. Alignment to Problem Statement

Requirement (from problemStatement.md)	Where addressed / iteration scope
Curated corpus	Phase 0 — corpus is exactly the 5 Groww HDFC scheme URLs (locked in sources.yaml)
3–5 schemes, category diversity	Phase 0 — 5 schemes: Mid Cap, Flexi Cap, Focused, ELSS, Large Cap
≤ 3 sentences, exactly 1 citation	Phase 3 (prompt contract + deterministic post-checks)
Footer “Last updated from sources:”	Phase 3 (post-processor)
Refuse advisory queries with educational link	Phase 3 — link returned is the matching Groww scheme URL (one of the 5)
Welcome msg, 3 examples, visible disclaimer	Phase 4 (UI)
No PII collection/storage	Phase 3 (PII guard) + Phase 5 (log scanner)
Source restriction	Phase 0 — sources.yaml contains only the 5 Groww URLs; Phase 5 CI fails on any other URL in output
Performance queries	Phase 3 — redirect to the relevant Groww scheme URL; no returns computation/comparison
Accuracy + auditability	Phase 5 (eval suites + structured PII-free logs)
Statement / capital-gains / KIM-only facts	Out of scope this iteration — assistant returns “I don’t have a verified answer” + Groww URL

Disclaimer Snippet (used in UI and every refusal):

Facts-only. No investment advice.

Prompts

1. Generate a detail phase wise architecture for @docs/problemStatement.md and save it in docs folder
2. In @docs/architecture.md in phase0 use these URLs for this project -

<https://groww.in/mutual-funds/hdfc-mid-cap-fund-direct-growth>

<https://groww.in/mutual-funds/hdfc-equity-fund-direct-growth>

<https://groww.in/mutual-funds/hdfc-focused-fund-direct-growth>

<https://groww.in/mutual-funds/hdfc-elss-tax-saver-fund-direct-plan-growth>

<https://groww.in/mutual-funds/hdfc-large-cap-fund-direct-growth>

3. Currently for this project we will pick exactly these URLs and not other URLs
4. In docs add a edge case file for each phase as per the @docs/architecture.md
5. Implement phase0 as per the @docs/architecture.md Keep proper folder structure for all the phases
6. In @docs/architecture.md in phase1 add subphases as we will try to implement each subphase one by one. Update only in @docs/architecture.md
7. Implement phase 1.1 as per the @docs/architecture.md
8. In ingestion store the code of phase1.1 in proper folder and create proper structure for other subphases and run phase1.1
9. Implement phase 1.2 and 1.3 as per the @docs/architecture.md
10. update docs/architecture.md Phase 1.4 to match the realities of this data
11. Chunking strategy in phase 1.4 in the @docs/architecture.md based on the current data is correct ?
12. Implement phase 1.4 as per the @docs/architecture.md
13. Based on the data does bge-small-en work in this scenario for phase 1.5 in the @docs/architecture.md
14. Implement phase 1.5 as per the @docs/architecture.md and run it as well
15. In which phase we are storing embedding in vector DB or chroma DB ?
16. Implement phase 1.6 as per the @docs/architecture.md
17. For scheduler and getting the latest data everytime we can use github action and we can update it in the @docs/architecture.md
18. Implement phase 1.7 as per the @docs/architecture.md
19. What is the best retrieval strategy we can use for phase 2 based on current data we have ?
20. Update this strategy in @docs/architecture.md and implement phase 2

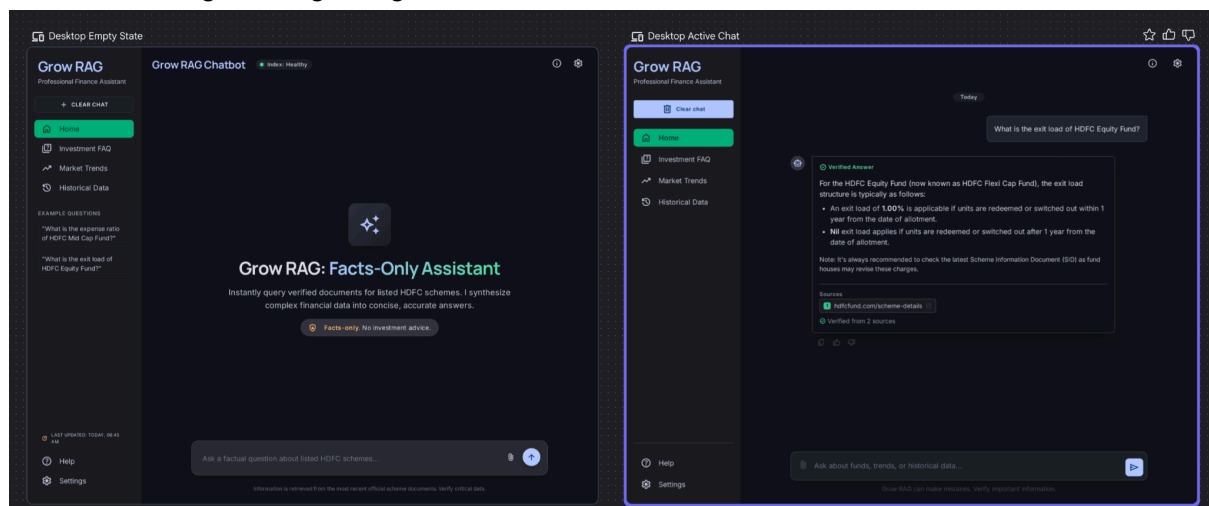
21. In phase3 if we don't know the answer to the query we will not attach any URL for anything with personal information we don't want. Update this in @docs/architecture.md and implement phase 3 and run it
22. LLM used in this project would be groq update it
23. In phase3 have we implemented LLM (Groq) Integration ?
24. In phase 3 we can add LLM (Answer Generator) Groq
25. Create an env file for API key
26. Implement backend for phase4 as per the @docs/architecture.md once backend is complete then frontend we can build
27. Implement phase4 frontend as well per the @docs/architecture.md
28. Run both backend and frontend

Updating the frontend

Gave cursor this prompt →

- Generate a prompt that I give to Google stitch for generating dark theme UI for this Grow RAG chat bot

Generated Images using Google stitch



Update the frontend using @stitch_grow_rag_chat_ui/

I have generated the design using the above prompt you shared

SCHEDULER Testing Prompts →

run scheduler locally so that we can test if entire scheduler is working up also can we check if github actions is configured properly

Check if everything in scheduler for github actions is correct or not and update it to run at 10:00 AM IST

Push the project on github - https://github.com/saksham20189575/m2_2.git

NEXT STEPS

- Push the project to github
- Testing of scheduler component → Github actions
- Deployment

Deployment

Option 1 → Backend + Frontend → Streamlit

Option 2 →

Backend → Streamlit or **Railway**

Frontend → Vercel

Docker → **Container (Black Box** → **Dependencies of project)**

Log → for debugging the scheduler locally you can add

Scheduler → Github action

Go to actions and trigger it

